V1.85 RC4 PQC validation — pqctoday-tpm x wolfTPM v4.0.0

Cross-implementation interop report and acknowledgement of wolfSSL / wolfTPM support

pqctoday-tpm maintainers

2026-05-15

Contents

Executive summary

Two independent V1.85 RC4 PQC TPM stacks now agree byte-for-byte on every PQC operation in the spec:

- **pqctoday-tpm** — libtpms v0.10.2 fork + swtpm v0.10.1, OpenSSL 3.6.2 EVP backend
- **wolfTPM v4.0.0** — PR #445 merge `fbbf6fe`, wolfCrypt backend

181 PASS / 0 FAIL across three validation layers (reference-document audit, single-implementation unit testing, two-implementation runtime cross-check). wolfTPM is the authoritative reference client our suite measures against. This document is both a validation report and an acknowledgement of the upstream work and review rigor that made it possible.

Scope at a glance

Layer	Question answered	Second implementation in the loop?	PASS / total
1. Reference documents	Is the *spec text* we cite real and current?	n/a — manual PDF grep + archive	n/a (process)
2. Unit / source-level	Does *our* libtpms match the spec when read in isolation?	No — OpenSSL EVP only, no socket	**127 / 127**
3. Runtime cross-check	Do *both* TPM stacks emit identical bytes on the wire?	Yes — wolfTPM v4.0.0 + wolfCrypt	**54 / 54**
Total	—	—	**181 / 181**

The three layers are deliberately distinct. Layer 1 fixes the citation foundation. Layer 2 catches single-implementation regressions cheaply. Layer 3 catches cross-implementation divergences that source-level checks structurally cannot see — and every one of our v0.8.0 wire-format fixes originated in Layer 3.

Layer 1 — Reference documents

The published standards we validate against. All are archived under `docs/standards/` (TCG) or `tests/compliance/vectors/` (NIST) at the commit this report is cut from, so every citation in Layers 2–3 is reproducible without external fetches.

Standard	Edition / date	Coverage in this report
TCG TPM 2.0 Library Specification	**V1.85 RC4** (2025-12-11), Parts 0–3	PQC commands, types, parameter sets, wire format
TCG EK Credential Profile	**V2.7 RC1**	PQC EK templates (Tables 13/14), §5.3.1 NV cert slots, §6.2.x OIDs
NIST FIPS 203	August 2024	ML-KEM-512 / 768 / 1024 sizes and KAT
NIST FIPS 204	August 2024	ML-DSA-44 / 65 / 87 sizes and KAT
NIST CSOR OIDs	Current	EK cert SPKI `AlgorithmIdentifier` byte-match vectors
NIST ACVP test vectors	`internalProjection.json` for ML-DSA keyGen	75 known-answer tests, Layer 2

Citation rule. Every spec reference in code, tests, and docs must resolve to an exact section / table number that PDF-greps positive against the archived edition. This rule was tightened on 2026-05-15 after a self-audit fixed 26 wrong V1.85 RC4 citations that had propagated from an early LLM enrichment pass (see “What wolfSSL / wolfTPM contributed”, item 3).

Layer 2 — Unit / source-level testing

Single-implementation assertions about pqctoday-tpm in isolation. No second crypto stack, no socket, no wolfTPM. Catches drift between our source and the reference documents from Layer 1.

Suite	Coverage summary	PASS / total
Source-level V1.85 RC4 + V2.7	58 named sections of grep-style spec checks	**106 / 106**

Suite	Coverage summary	PASS / total
Phase 3 in-process crossval	11 TPM commands via direct `TPMLIB_Process`, no socket	**21 / 21**
NIST ACVP ML-DSA keyGen KAT	75 canonical FIPS 204 keyGen vectors (in source-level total)	included
Layer 2 total	—	**127 / 127**

Layer 2 — suite detail

`tests/compliance/v185_compliance.sh` — 58 named sections covering: algorithm IDs (Part 2 §6.3), parameter set IDs (§11), FIPS 203 / 204 sizes (§14 / §15), HashML-DSA domain-separation context (§10), I/O buffer sizing for ML-DSA-87 (§9), V1.85 PQC command codes (§13 Table 11), algorithm-enable flags, TPMU union extensions (Tables 184 / 189 / 195), V1.85 new type definitions (Tables 99-101, 110-112, 208, 216-221), V2.7 EK constants, and the OpenSSL 3.6+ provider surface our crypto backend depends on.

`tests/crossval/src/test_pqc_phase3.c` — TPM commands driven via direct `TPMLIB_Process` (no swtpm socket, no wolfTPM): `TPMLIB_MainInit` with file-backed NV, `TPM2_Startup(CLEAR)`, `TPM2_CreatePrimary(ML-KEM-768 EK)`, `TPM2_CreatePrimary(ML-DSA-65 restricted+sign AK)`, `TPM2_MakeCredential` + `TPM2_ActivateCredential` roundtrip over the ML-KEM-768 EK, restricted-key `TPM2_SignDigest` ticket gate (§20.7.1), `TPM2_CreatePrimary(ML-DSA-65 unrestricted)`, unrestricted `TPM2_SignDigest` roundtrip, and a permanent V1.85 RC4 SignDigest wire-conformance gate (Test 11) added 2026-05-15.

`tests/crossval/src/kat_loader.c` + `test_pqc_crossval.c` — NIST ACVP `internalProjection.json` parser. For each of 75 ML-DSA keyGen vectors: expand 32-byte ξ seed via `EVP_PKEY_fromdata(OSSL_PKEY_PARAM_ML_DSA` derive the public key, and byte-compare against the canonical `pk` from the vector. Gold-standard FIPS 204 compliance proof for the OpenSSL EVP path; counted inside the source-level total above.

What Layer 2 does NOT prove. That two independent TPM implementations produce identical bytes on the wire. Layer 2 will pass even if our marshaller and our unmarshaller agree with each other and both diverge from the spec — that is exactly the failure mode Layer 3 is designed to catch (and did catch, four times — see "Where pqctoday-tpm landed").

Layer 3 — Runtime cross-check vs wolfTPM v4.0.0

Two independent TPM stacks driven against each other over the Microsoft TPM simulator socket protocol. The wolfTPM v4.0.0 client (PR #445, `fbbf6fe`) is the reference; our swtpm-backed libtpms is the system under test. Wire bytes are produced by wolfCrypt on one side and accepted (and re-emitted) by OpenSSL EVP on the other. Every PASS here is a byte-level interop assertion between two crypto stacks that share zero source code.

Suite	Coverage summary	PASS / total
Runtime cross-check	ML-KEM Encap/Decap + ML-DSA full sign/verify + SignDigest wire gate	**30 / 30**
Attestation cross-check	`TPM2_Quote` + `TPM2_Certify` with ML-DSA AK, dual-verified	**12 / 12**
V2.7 EK template conformance	Six V2.7 RC1 PQC EK templates byte-exact accepted	**6 / 6**
V2.7 EK cert NV-slot conformance	Six §5.3.1 NV cert slots with NIST CSOR OID byte-match	**6 / 6**
Layer 3 total	—	**54 / 54**

Layer 3 — suite detail

`tests/compliance/run_wolftpm_runtime_xcheck.sh` — full ML-KEM-{512, 768, 1024} `Encapsulate` / `Decapsulate` round-trip (Part 3 §14.10 / §14.11); full ML-DSA-{44, 65, 87} sign + verify roundtrip via `SignSequenceStart` / `SequenceUpdate` / `SignSequenceComplete` / `VerifySequenceStart` / `VerifySequenceComplete` (§17.5 / §20.6 / §20.3), with the resulting `TPMT_TK_VERIFIED` ticket carrying `tag = TPM_ST_MESSAGE_VERIFIED` per Table 119; plus `wolfTPM/examples/pqc/pqc_mssim_e2e` driving

`wolfTPM2_SignDigest` and `wolfTPM2_VerifyDigestSignature` against V1.85 RC4 Table 126 / Table 120 wire shapes.

`tests/compliance/run_attestation_xcheck.sh` + **`clients/pqc_attestation_xcheck.c`** — `TPM2_Quote` and `TPM2_Certify` driven against our TPM with ML-DSA-44 / 65 / 87 attestation keys. Each signed attestation blob is captured on the wire and *independently* verified by both wolfCrypt (FIPS 204 reference path) and OpenSSL EVP. Dual-verification means a PASS proves the bytes leaving our TPM are valid FIPS 204 signatures any V1.85 PQC verifier will accept.

`tests/compliance/run_ek_conformance_xcheck.sh` — wolfTPM marshals the six V2.7 RC1 PQC EK templates (Tables 13 / 14) on the client side and submits `TPM2_CreatePrimary` to our libtpms; our TPM accepts them byte-exact and produces the FIPS 203 / 204 public-key sizes in `outPublic.unique` (800 / 1184 / 1568 B for ML-KEM, 1312 / 1952 / 2592 B for ML-DSA).

`tests/compliance/run_ek_cert_conformance_xcheck.sh` — for each of six V2.7 §5.3.1 NV indices (`0x01c00060/62/64/70/72/74`): `TPM2_NV_ReadPublic` and `TPM2_NV_Read` retrieve the cert; OpenSSL `d2i_X509` parses it; the SPKI `AlgorithmIdentifier` OID body byte-matches the NIST CSOR reference (`id-alg-ml-kem-{512, 768, 1024}` / `id-ml-dsa-{44, 65, 87}`); the cert validity window is sane.

Layer 3 sample output

```
=== ML-KEM Encap/Decap roundtrip (§14.10 / §14.11) ===
  [PASS] ML-KEM-512   pubkey  800 B / ct  768 B / ss 32 B - round-trip OK
  [PASS] ML-KEM-768   pubkey 1184 B / ct 1088 B / ss 32 B - round-trip OK
  [PASS] ML-KEM-1024  pubkey 1568 B / ct 1568 B / ss 32 B - round-trip OK

=== ML-DSA full sign+verify roundtrip (§17.5 / §20.6 / §20.3) ===
  [PASS] ML-DSA-44    pubkey 1312 B / sig 2420 B
                      VerifySequenceComplete -> TPM_ST_MESSAGE_VERIFIED (Table 119)
  [PASS] ML-DSA-65    pubkey 1952 B / sig 3309 B
                      VerifySequenceComplete -> TPM_ST_MESSAGE_VERIFIED (Table 119)
  [PASS] ML-DSA-87    pubkey 2592 B / sig 4627 B
                      VerifySequenceComplete -> TPM_ST_MESSAGE_VERIFIED (Table 119)

=== V1.85 RC4 SignDigest / VerifyDigestSignature (§20.7 / §20.4) ===
  [PASS] wolfTPM/examples/pqc/pqc_mssim_e2e - MLKEM Encap/Decap + HashMLDSA-65
         SignDigest/Verify all green against V1.85 RC4 Table 126 / Table 120
```

What wolfSSL / wolfTPM contributed

1. PR #445 — authoritative V1.85 PQC client

wolfSSL/wolfTPM#445 (Aidan Garske, merged 2026-04-29) is the V1.85 PQC support that ships in wolfTPM v4.0.0. Throughout our Phase 2–4 work it has been the reference client we pointed every wire-format question at. None of the four mismatches below were spec-clear from a one-implementation read of the PDF; wolfTPM's wire output put each one under our nose.

1. `TPMS_MLDSA_PARMS.allowExternalMu` (Part 2 §12.2.3.6 Table 229) — wolfTPM marshalled 3 bytes; we expected 2. `CreatePrimary` returned `TPM_RC_SIZE` (`0x2D5`). Added the field; commit `ea52cf9d`.

2. `TPMS_MLKEM_PARMS.symmetric` ordering (Part 2 §12.2.3.8 Table 231) — wolfTPM emits `TPMT_SYM_DEF_OBJECT` *before* `parameterSet`; our libtpms expected `parameterSet` only. `CreatePrimary` returned `TPM_RC_VALUE` (`0x2C4`). Added the field in spec order; commit `ea52cf9d`.

3. `TPM2_Encapsulate` response shape (Part 3 §14.10 Table 61) — spec orders `{sharedSecret, ciphertext}`; we had it reversed. `Decapsulate` returned `TPM_RC_SIZE` (`0x95`). Reordered; commit `23a718f6`.

4. `TPM2_SignDigest` / `TPM2_VerifyDigestSignature` wire (Part 3 §20.7 Table 126 / §20.4 Table 120) — `wolfTPM2_SignDigest` returned `rc=0x1d2 = TPM_RC_SCHEME`. We were on the pre-RC4 working-draft `{inScheme, digest, context, hint}`; RC4 is `{keyHandle, context, digest, validation}`. v0.8.0 wire migration in commit `e4f83a61`. `pqc_mssim_e2e` is now a permanent Layer 3 regression gate.

2. Issue #499 / PR #501 — build-system fix

wolfSSL/wolfTPM#499 (we filed 2026-05-11) — `configure.ac` probed the wrong wolfSSL header (`mlkem.h` instead of `wc_mlkem.h`). PR #501 (Aidan Garske) merged the same day with a bonus wolfSSL version-matrix CI workflow (`v5.8.0-stable / v5.9.1-stable / master`). We dropped our Dockerfile workaround and re-ran the cross-check at the new pin: 29 PASS / 0 FAIL without the symlink. Detailed write-up in `wolfTPM-001-mlkem-header-rename.md`.

3. Issues #504 / #505 / #506 — review rigor that drove our citation audit

We filed three tickets on 2026-05-13 alleging structural gaps in wolfTPM's V1.85 implementation. **Aidan closed all three as not-bugs** with verbatim-grep refutations against the V1.85 RC4 PDF.

#504 — Our claim: `TPMU_SIG_SCHEME` missing `mldsa` / `hash_mldsa` arms, cited Table 216. Correction: Table 216 is `TPM2B_SIGNATURE_MLDSA`. `TPMU_SIG_SCHEME` is Table 181 (§11.2.1.4 p.173). The spec defines no `mldsa` / `hash_mldsa` arms — ML-DSA uses `TPM_ALG_NULL` as the scheme selector because `TPMS_MLDSA_PARMS` fully fixes the operation.

#505 — Our claim: `TPMT_TK_VERIFIED` defined as an empty struct. Correction: the struct has always carried `{tag, hierarchy, [V185 metaAlg], digest}` — never empty. Our "current state" snippet was an LLM fabrication.

#506 — Our claim: `TPM2B_DIGEST_INFO` / `TPM2B_MU` typedefs missing for §29.2.1 `TPM2_SignDigest`. Correction: `TPM2_SignDigest` is §20.7; §29.2.1 is `TPM2_ClockSet`. Neither typedef exists anywhere in V1.85 RC4. External-μ rides in the existing `TPM2B_DIGEST digest` field per Table 126, gated by `TPMS_MLDSA_PARMS.allowExternalMu`.

The closures triggered a same-day self-audit on our end. **26 wrong V1.85 RC4 citations** had propagated across our code, tests, and "spec-authoritative" doc extract — clustered in two failure modes: PQC commands stamped as §29.x (the `TPM2_Clock*` chapter) because an early LLM enrichment placed PQC in a non-existent §29 PQC chapter; and §18 command Tables in `TPMdocextract.md` cited Table numbers from the §17 Policy-command range. All 26 fixed in commit `eb0e3c9b`.

We then wrote a permanent internal process rule: **no upstream issue alleging a missing structure, wrong field, or incomplete implementation may be filed without (a) PDF grep against `docs/standards/` confirming the exact table / section number, (b) reading the vendor source at `vendor/wolftpm/wolftpm/tpm2.h` as-is, (c) cross-check against `docs/TPMdocextract.md`.** Three closed tickets gave us a far better return on investment than three "real" bugs would have, because they fixed a process defect rather than a code defect.

4. wolfTPM examples and wolfCrypt as a second verifier

`examples/pqc/mldsa_sign`, `examples/pqc/mlkem_encap`, and `examples/pqc/pqc_mssim_e2e` are wired into our CI as regression gates. Each one drove a specific Phase 3.5 / Phase 4 / v0.8.0 fix and each one remains a load-bearing Layer 3 step. We did not have to write any of these clients — wolfTPM shipped them ready to drive a swtpm-backed libtpms over the Microsoft TPM simulator socket.

wolfCrypt as the *independent* verifier in the attestation cross-check is what makes Layer 3's 12 attestation PASS results load-bearing rather than circular: signatures are produced via OpenSSL EVP and verified via wolfCrypt's FIPS 204 reference path. Two crypto stacks, zero shared source code, byte-identical agreement.

Where pqctoday-tpm landed

pqctoday-tpm is the open-source TPM 2.0 emulator side of the same V1.85 cross-validation matrix that wolfTPM occupies on the client side. We took libtpms v0.10.2 + swtpm v0.10.1 and added:

- All **eight V1.85 PQC commands**: `TPM2_Encapsulate`, `TPM2_Decapsulate`, `TPM2_SignSequenceStart` / `Complete`, `TPM2_VerifySequenceStart` / `Complete`, `TPM2_SignDigest`, `TPM2_VerifyDigestSignature`.
- ML-DSA-44 / 65 / 87 and ML-KEM-512 / 768 / 1024 backed by **OpenSSL 3.6.2 EVP** (independent crypto stack from wolfCrypt — this is what makes Layer 3 meaningful).
- Six **V2.7 RC1 PQC EK templates** and six §5.3.1 NV cert slots, byte-exact against TCG EK Credential Profile V2.7 RC1 Tables 13 / 14 with NIST CSOR OID-bearing SPKIs.
- **Phase 4.1 auth-area integration** for a parallel sequence-handle pool in the vendor range `0x80FF0000`–`0x80FF00FF` (eight functions across `Object.c`, `Entity.c`, `SessionProcess.c`).
- An **Emscripten / WASM build target** (`wasm/dist/pqctpm.{js,wasm}`) that runs the entire V1.85 PQC + V2.7 EK Credential Profile surface in a browser, with a JS-side PQC bridge that delegates the FIPS 203 / 204 primitives to the host's PQC engine.

The Phase 3.5 conformance fixes, the §14.10 Table 61 Encapsulate response reorder, and the v0.8.0 SignDigest wire migration all originated as Layer 3 cross-impl mismatches against wolfTPM v4.0.0. None of them would have been catchable from Layer 2 alone.

Reproduction

```
git clone --depth=1 https://github.com/pqctoday-org/pqctoday-tpm
cd pqctoday-tpm
docker build -f docker/Dockerfile.xcheck -t pqctoday-tpm-xcheck .
docker run --rm -v "$PWD:/workspace" -w /workspace pqctoday-tpm-xcheck bash -c '
  # Layer 2 — single-implementation
  bash tests/compliance/v185_compliance.sh                  # 106 / 106
  make crossval                                              #  21 / 21 (+ NIST ACVP KAT)

  # Layer 3 — cross-implementation
  bash tests/compliance/run_wolftpm_runtime_xcheck.sh        #  30 / 30
  bash tests/compliance/run_attestation_xcheck.sh            #  12 / 12
  bash tests/compliance/run_ek_conformance_xcheck.sh         #   6 /  6
  bash tests/compliance/run_ek_cert_conformance_xcheck.sh    #   6 /  6
'
```

`Dockerfile.xcheck` pins `WOLFTPM_REF=0ae18dcd138f2ea2aba7d146d05115cf294c07bb` (PR #501 merge — the wolfSSL header-rename fix) and builds wolfSSL `master` with `--enable-experimental --enable-dilithium --enable-mlkem`. The same matrix runs daily as the `xcheck.yml` GitHub Actions workflow against three wolfSSL versions.

Thanks

To **Aidan Garske** for PR #445, for the rapid #499 / #501 turnaround, and most of all for the patient, PDF-grounded refutations on #504 / #505 / #506. Those three closed tickets did more to harden our V1.85 process than three "real" bugs would have. The same-day citation audit they triggered is the reason v0.8.0 ships clean.

To **David Garske** and the wolfSSL team for shipping `--enable-experimental --enable-dilithium --enable-mlkem` years ahead of the FIPS 203 / 204 finalization. Without a production-quality second crypto stack to cross-verify against, "compliance" against the V1.85 RC4 spec would have been a single-implementation echo chamber.

To **wolfTPM as a project** for shipping a complete PQC TPM client — eight commands, three signing flows, three KEM flows, three EK Credential Profile generations — in a single PR. It is the reference implementation we measure everything else against.

The pqctoday-tpm side of the matrix is BSD-3-Clause at pqctoday-org/pqctoday-tpm. Every Layer 3 assertion reproduces in a single Docker invocation; please pull, run, and challenge any of them. We will treat a divergence report from wolfTPM the same way we now treat a wolfTPM #504 / #505 / #506 closure — as the most useful possible signal from the spec implementer with the better-grounded read.

— *pqctoday-tpm maintainers, 2026-05-15*